Boundary Case: Vendor Data-Handling-Policy Change as Standalone Architectural Treatment — Formalizing How CKS Distinguishes Vendor-Level Constraints from Substrate-Resident Authoritative Rules, Treating Vendor Policy Changes as External Signals That Trigger Human-Authored Rule Revisions Without Auto-Compliance

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one boundary case — the vendor data-handling-policy change boundary — as a standalone architectural treatment, articulating how a CKS-pattern deployment distinguishes vendor-level constraints from substrate-resident authoritative content and treats vendor policy changes as external signals that trigger human-authored rule revisions.

Abstract

A CKS-pattern deployment typically depends on vendors for substrate hosting, AI consultation, or other architectural functions, and those vendors periodically change their data-handling policies. Such changes — terms-of-service revisions, regulatory changes affecting the vendor (data-residency, sector-specific obligations), retention or training-data revisions, geographic restrictions, and certification changes — are operationally common and often consequential. They stress a distinction the source paper draws but does not name as a boundary case: between vendor-level constraints and substrate-resident authoritative rules. Vendor constraints may force operational adjustments, but they are not authoritative content for the deployment; substrate-resident rules are. This note formalizes the boundary as a standalone architectural treatment, identifies the commitments stressed, states the treatment that resolves the boundary, enumerates the anti-pattern treatments that violate the architecture, and bounds the treatment's scope.

1. Why the boundary needs to be formalized as standalone

A CKS-pattern deployment relies on host infrastructure provided by vendors: substrate storage, the mediator LLM, and adjacent components such as search, indexing, observability, and identity. The source paper's tool-agnosticism commitment (§7.1) makes vendor portability a property of the architecture, not an aspiration. It does not exempt deployments from the operational reality that vendors revise their policies. Terms-of-service revisions are routine; regulatory regimes evolve continuously; vendors revise retention policies, modify training-data terms, restrict or expand geographic availability,

and change security certifications. Each change is a signal that the deployment must process.

The natural-language readings of "must process" sit close to architectural anti-patterns. *Comply with the new policy*, *update the deployment to match the policy*, and *adapt automatically to vendor changes* each slide easily into letting vendor policy determine substrate behavior, placing a vendor-revocable artifact where the architecture reserves authoritative content, or removing the human governance moment the architecture commits to. The slides are invisible at each step; the resulting system passes casual review while violating the architecture at its load-bearing seam.

This is what makes the boundary case worth standalone formalization. The connection to §11.4's authority test is direct: the test asks which artifact governs cell behavior when artifacts disagree, and the vendor data-handling-policy change boundary is where deployments most often fail it without noticing. The strategic prior-art posture matters because "AI compliance" and "vendor governance" are dominant framings in 2024–2026 commercial discourse, and many of the patterns sold under those headings are exactly the anti-patterns this note names. A6.09 is the ninth Phase A6 boundary case note; subsequent notes cover cell timeout / long-running operation (A6.10), substrate concurrent-write race (A6.11), and additional cases.

2. The boundary case scenario and what makes it non-obvious

A vendor providing some architectural function publishes a data-handling-policy change with a stated effective date and possibly compatibility, transition, or grandfathering provisions. The deployment must respond, and the response will be observable in behavior, content, and rules going forward. Three properties of the scenario combine to make the boundary easy to misread.

The signal is upstream and external. The vendor change does not arrive through the substrate's normal write paths. It arrives through email, dashboard notice, version-controlled documentation, regulatory bulletin, or vendor support communication. None of those paths is a CKS-coherent rule-authoring path; an artifact arriving through them is informational, not authoritative.

The signal is plausibly directive. Vendor policy changes are often phrased in directive terms ("you must," "users are required to," "by continuing to use the service") and may include guidance that reads like step-by-step instructions. The directive phrasing is a property of the vendor's communication style, not a grant of authority over the deployment. Treating directive phrasing as if it carried governance weight is the central misreading.

The signal is operationally pressing. Effective dates create time pressure. Pressure tempts deployments to skip the human governance moment in favor of automated compliance, and tempts implementers to wire vendor notices directly into substrate behavior on the grounds that "we'll have to comply anyway." The fact that compliance will likely be required does not change which artifact carries authority.

The combination of upstream-and-external, plausibly-directive, and operationally-pressing makes the boundary stress the architecture rather than trace through it cleanly.

3. Which architectural commitments are stressed

Five commitments are stressed by the boundary.

Substrate-as-source-of-truth and the test of authority (§11.3, §11.4). The source paper commits the substrate to carrying authoritative content and commits the architecture to a test asking which artifact governs cell behavior when artifacts disagree. A vendor policy and a substrate-resident rule can disagree. The test resolves it: the substrate-resident rule governs, until a human authors a rule revision.

Tool-agnosticism (§7.1, formalized standalone in A1.05). The commitment that the deployment is portable across host environments meeting the three minimal requirements makes vendor migration a property of the architecture rather than a contingency. A vendor change incompatible with the deployment's requirements is the case tool-agnosticism was specified for.

Rule authoring as the governance moment (§3.3, §6.3, formalized standalone in A2.04). Rule authoring is one of the two moments at which human governance is exercised. The substrate's response to a vendor policy change is, architecturally, a rule-authoring decision: humans decide what the deployment will do, and record the decision as a substrate-resident rule.

Human-governance as authority architecture, not procedural promise (§3.3, formalized standalone in A1.01). The rights to inspect, modify, and override are properties of the system's design — not features of a vendor's current policy. If vendor policy could in principle prevent humans from inspecting, modifying, or overriding substrate content or rules, the architecture would have failed before the policy change arrived.

Vendor-independent authoritative content (A4.10). Authoritative content is determined by substrate-resident rules under human authority, not by any vendor's policy. The boundary case is exactly the scenario A4.10 was specified for: it stress-tests whether authoritative content remains substrate-resident when vendor policy changes attempt, by phrasing or operational pressure, to assume that role.

The five are not new commitments; the boundary stresses them in combination, and the standalone treatment names what holding the combination together looks like.

4. The architectural treatment

The treatment has six components.

(a) Vendor policy changes are external signals, not authoritative content. When a vendor publishes a policy change, the artifact that arrives is informational. It describes what the vendor will do or require; it does not, by virtue of arrival, govern cell behavior. It is a signal that triggers a governance moment, not a directive that bypasses one.

(b) Humans evaluate the signal through the inspect right. A human with appropriate access reads the vendor change, reads the substrate-resident rules potentially affected, and assesses the relationship — a governance activity (A1.01) exercising the inspect right (A2.01). LLM-assisted summarization is permissible adjacent tooling; it is not a governance step.

(c) Substrate response is human-authored rule revision per A2.04. Whatever the deployment will do is encoded as a substrate-resident rule, authored by humans. The rule may revise, replace, or add a rule, or — in the migration case — direct the deployment toward an alternate vendor. The human-authored rule governs going forward; the vendor's policy text does not.

(d) Compatible-with-adjustment changes follow the rule-revision path. When the vendor change is operationally compatible with continued deployment after some rule adjustment (a retention period changes; a residency requirement is satisfiable by an addable region; a certifications change requires updated provenance metadata), the response is a rule revision following A6.02 retroactivity treatment: the new rule version applies forward; existing substrate content carrying the prior version retains it as recorded provenance (A1.07 / A2.40); cell behavior continues per the most recent applicable version.

(e) Incompatible changes follow the migration path per A1.05 and A6.03. When the vendor change is operationally incompatible with continued deployment, the deployment moves to an alternate vendor satisfying tool-agnosticism's three minimal requirements. The migration sequence — substrate state, rule, provenance, and addressability preservation — follows A6.03; A6.09 differs from A6.03 in trigger (policy change versus availability loss) but converges on the same pattern.

(f) Vendor policies may be mirrored as informational substrate content with provenance, but mirroring does not make them authoritative. The deployment may record the published policy with full A2.40 provenance. This is the source-of-truth-vs-mirror distinction (A2.48): the substrate carries mirrored policy as informational content; it carries the human-authored response rule as authoritative content. Cell behavior continues per substrate-resident rules even when mirrored vendor text suggests otherwise. The history of the response — vendor change, evaluation, rule revision or migration directive, resulting state changes — is recorded with A2.40 provenance fields per A1.07.

5. Anti-pattern treatments that would violate the architecture

Seven anti-pattern treatments name the most common ways a deployment can fail the boundary while appearing to "respond" to a vendor change.

Vendor-policy-as-rule. Treating the vendor's policy text as authoritative content — as if the vendor's words substituted for human-authored substrate-resident rules. This is a canonical instantiation of vendor-revocable governance (A3.01).

Auto-compliance-with-vendor-changes. Wiring vendor notices directly into substrate behavior so that policy changes propagate to operational behavior without a human governance moment. The architecture's response requires a governance moment by construction; auto-compliance removes it, which removes the architecture's load-bearing property.

LLM-mediated-vendor-policy-evaluation. Delegating evaluation of vendor changes and authoring of rule revisions to an LLM operating outside the human-governance authority architecture. This is an instantiation of A3.13. The architecture admits LLM-drafted rule proposals subject to human authority before they take effect; it does not admit LLM-committed rule revisions outside human authority.

Compliance-framework-driven-rule-revision. Letting an external compliance framework, audit tool, or vendor-management product author rule revisions on the basis of vendor changes, without human governance. This is a structural variant of vendor-policy-as-rule. Compliance frameworks are at most informational inputs to a governance moment.

Vendor-policy-overrides-substrate-rule. Treating vendor policy as architecturally superior to substrate-resident rules in cases of disagreement. The architectural test of authority (§11.4) resolves the disagreement in the opposite direction.

Silent-vendor-adaptation. Allowing substrate behavior to change in response to vendor policy without recording the change as a rule revision with provenance. This violates path retraceability (A1.07): a future inspector cannot determine, from the substrate alone, why behavior changed.

Vendor-controlled-training-data-overrides-rule. Letting the vendor's training-data policy determine what data the substrate stores or omits, in place of substrate-resident rules per A2.04. Vendor training-data terms are inputs to rule design, not substitutes for the rules themselves.

6. Operational implications

Three operational implications follow.

Vendor changes are governance moments by construction. A deployment instantiating the treatment encounters every vendor data-handling-policy change as a governance moment: a human reads, evaluates, decides, and authors. This is not incidental overhead; it is the architecture's load-bearing property at the seam where vendor authority and deployment authority can collide.

Authority is preserved across vendor turnover. Because authoritative content is substrate-resident and rules are human-authored, vendor changes — including changes that lead to migration — do not change *who governs the deployment* or *what authoritative content the deployment carries*.

Path retraceability captures the response history. Every governance moment triggered by a vendor change leaves a trace readable through the inspect right at any future time. A regulator, auditor, or successor team can reconstruct the deployment's history of vendor responses without depending on any vendor's continued cooperation.

7. Limits of the architectural treatment

The treatment applies specifically to *vendor-level* data-handling-policy changes — changes published by a vendor that supplies an architectural function. It does not apply to four adjacent classes. **Internal organizational policy changes** are authored directly into substrate-resident rules per A2.04, without the external-signal step. **Direct regulatory changes** are inputs to substrate-resident rule revisions; the path is §4's, but migration (A1.05) is generally unavailable since regulators cannot be migrated from. **Composition partner internal changes** are handled by the composition pattern's contract per A6.05. **Vendor unavailability** without a policy-change trigger is A6.03's boundary; the two converge on migration but differ in trigger and evaluation activity.

Two operational limits bear naming. Migration is architecturally available because A1.05 commits the deployment to portability across environments meeting the three minimal requirements; it is not frictionless, and may be operationally constrained by jurisdiction, schedule, or budget. Architectural availability guarantees that migration is not precluded, not that it is cheap or fast. Separately, the treatment does not adjudicate between vendor interpretations of ambiguous policies; interpretation is a governance input.

8. Operational test

The architectural treatment is satisfied by a deployment if and only if all of the following are true at all times during the substrate's existence:

1. Vendor data-handling-policy changes do not directly modify cell behavior; behavior change requires a substrate-resident rule revision authored by a human under A2.04.
2. Vendor policy text, when recorded in the substrate, is recorded as informational content with the source-of-truth-vs-mirror distinction (A2.48) preserved; it does not become authoritative content (A2.46) by virtue of recording.
3. Substrate-resident rules govern cell behavior even when mirrored vendor policy text suggests different behavior, until human-authored rule revision changes what the substrate carries.
4. Rule revisions in response to vendor changes follow A6.02 retroactivity: new versions apply forward; prior provenance is preserved per A1.07 / A2.40.
5. Migration in response to incompatible changes follows A6.03 patterns and tool-agnosticism's three minimal requirements (A1.05).
6. The history of the response is retraceable through the inspect right per A1.07.
7. No vendor policy artifact, no compliance framework, and no LLM operation outside human authority can in principle commit substrate-resident rule revisions on its own.

A deployment that fails any of (1)–(7) does not implement the architectural treatment, even if it satisfies other CKS commitments.

9. Conclusion

Vendor data-handling-policy changes are operationally common, plausibly directive, and operationally pressing. The combination tempts deployments into anti-patterns that quietly transfer authority from substrate-resident rules to vendor policy text — a transfer that often passes casual review and fails the architectural test of authority on inspection. Naming the boundary as standalone treatment specifies how the architecture distinguishes vendor constraints from substrate authority, what the legitimate response is (rule revision under A2.04 or migration under A1.05/A6.03), and which seven anti-pattern treatments name the typical ways a deployment fails the boundary.

The treatment preserves the source paper's central authority commitment under stress: the substrate is the source of truth (§11.3); substrate-resident rules govern cell behavior (§11.4); human governance is exercised at rule-authoring moments (§3.3, §6.3); the deployment is portable across vendors meeting the tool-agnosticism prerequisites (§7.1). Subsequent Phase A6 notes cover cell timeout / long-running operation (A6.10) and substrate concurrent-write race (A6.11).

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Boundary Case: Vendor Data-Handling-Policy Change as Standalone Architectural Treatment — Formalizing How CKS Distinguishes Vendor-Level Constraints from Substrate-Resident Authoritative Rules, Treating Vendor Policy Changes as External Signals That Trigger*

Human-Authored Rule Revisions Without Auto-Compliance. May 7, 2026. ORCID: 0009-0004-8065-3235.